

MemCatalog: A DBMS-style Memory Management System for LLM Agents

Anonymous Author(s)

Abstract

Large Language Model (LLM) agents accumulate interaction histories that grow to hundreds of thousands of tokens per user, yet current memory systems store this data but do not manage it: every question, whether a simple fact lookup or a count across multiple records, is served by the same retrieval pipeline. This leads to two limitations: the system fails systematically on complex queries that require aggregation or comparison, and it wastes LLM calls on easy queries that do not need them while never learning which memories are useful. We introduce MemCatalog, a memory management system for LLM agents that addresses three challenges of managing agent memory. (i) **Architecture**. MemCatalog organizes sessions into a typed catalog, routes each query to one of three patterns (point lookup, aggregation, or comparison) using keywords alone, and dispatches it to a specialized operator, with no LLM call needed for routing. (ii) **Performance**. A *Second-Order Memory* logs which catalog entries contribute to correct answers for each user and promotes useful entries in subsequent queries, forming a workload feedback loop that improves accuracy over time. (iii) **Efficiency**. A *Cost-Aware Executor* assigns different LLM call budgets to different query patterns and stops early when two samples agree. Across eight long-term memory benchmarks, MemCatalog matches or improves the strongest prior memory system on every one, by an average of 2.6 points and by up to 5.6 points on LME-s. On MemBench it further reduces LLM calls by 50.7% relative to a Uniform- $K=3$ reference while attaining higher accuracy.

CCS Concepts

• **Information systems** → **Data management systems**; **Query optimization**; • **Computing methodologies** → *Natural language processing*; Multi-agent systems.

Keywords

Agent memory, query optimization, large language models, self-tuning systems, cost-based optimization

ACM Reference Format:

Anonymous Author(s). 2026. MemCatalog: A DBMS-style Memory Management System for LLM Agents. In *Proceedings of Proceedings of the 2026 International Conference on Management of Data (SIGMOD '26)*. ACM, New York, NY, USA, 14 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '26, Bengaluru, India

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/06
<https://doi.org/XXXXXXX.XXXXXXX>

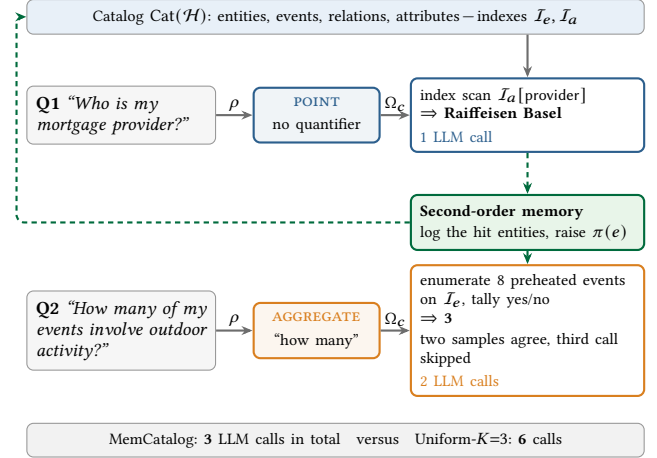


Figure 1: Motivating example. MemCatalog selects a physical plan for each query (**POINT** or **AGGREGATE**), serves it with the minimum LLM-call budget, and logs hit entities into second-order memory so that the next query reads them first. The interaction costs three LLM calls in total, where a Uniform- $K=3$ baseline would spend six.

1 Introduction

Memory is the foundation of a personal Large Language Model (LLM) agent. It allows the agent to remember a user's preferences, track promises and plans across conversations, and answer questions about the past. As agents are deployed in more settings, from personal assistants to customer service bots, the interaction history they accumulate grows rapidly, reaching hundreds of thousands of tokens per user, and in some deployments exceeding a million [7, 26, 29]. This growing volume makes it increasingly difficult to find the right information at the right time, and requires a systematic way to *manage* agent memory.

Yet today's memory systems store this data but do not manage it. Systems differ in what they store – raw transcripts [18], embedded chunks via Retrieval-Augmented Generation (RAG) [16], extracted facts [7], paged tiers [26], or graph edges [29] – but at query time every one follows the same pipeline: retrieve relevant passages, then generate an answer, regardless of what the query asks. No system distinguishes whether a query needs a simple fact lookup, an enumeration of multiple records, or a comparison across entries, and no system learns which stored memories are more useful than others. As Figure 1 illustrates, this leads to two limitations.

Two Limitations of Current Systems.

- **Static memory operations.** Every query goes through the same retrieval pipeline, and no system records which memories contributed to correct answers. As a result, systems fail on queries

that a single retrieval pass cannot serve, such as counting candidates across sessions or comparing values from distant passages [14], and every query starts from scratch as if the user had never asked anything before.

- **High cost of memory access.** No system differentiates queries by difficulty. A simple fact lookup that could be answered in one LLM call receives the same computational budget as a complex aggregation that benefits from multiple samples, wasting calls on easy queries while hard queries receive no extra allocation.

Both limitations stem from the same root cause: agent memory is treated as a flat store with no management layer. Database management systems solved the analogous problem for structured data decades ago: schemas organize what is stored, query planners choose how to answer each query, and cost models control how much computation each answer spends [12, 31]. Agent memory calls for the same separation of concerns, yet no existing system provides it. Building such a system raises three challenges.

Three Challenges for Memory Management.

- **Architecture: what to store and how to answer.** Agent memory is unstructured natural language. To manage it, the system needs to impose structure on the stored data, recognize different types of queries, and route each to specialized logic, ideally without spending an extra LLM call.
- **Performance: how memory retrieval improves over time.** A good memory system should get better as it serves more queries for the same user, just as a database improves its indexes as it collects workload statistics. This requires a feedback loop that records which stored entries helped answer past queries and uses this signal to reorganize the memory for subsequent queries, a mechanism that no existing agent memory system implements. The difficulty is that no ground-truth answers are available at deployment time, so the system must define “useful” without external supervision.
- **Efficiency: how to control the cost of memory access.** An LLM call is the dominant cost unit, and drawing multiple independent answers to the same query (Self-Consistency [36]) multiplies it. A system that spends the same budget on every query wastes calls on easy queries that do not need them, and cannot reallocate to hard queries that do. The system needs a cost model that assigns different budgets to different query types and stops spending when additional calls cannot change the outcome.

Our Proposal. To address these challenges, we propose MemCatalog, a memory management system for LLM agents that treats agent memory as managed data. As shown in Figure 3, MemCatalog consists of a framework and two optimization layers.

- **Framework: Typed Catalog, Query Planning, and Operators** (Section 4). To address the architecture challenge, MemCatalog organizes raw sessions into a typed catalog of entities, events, relations, and attributes. A lightweight planner recognizes three query patterns, POINT (fact lookup), AGGREGATE (aggregation), and COMPARE (comparison), by matching keywords in the query, with no LLM call. Each pattern is handled by a specialized operator that defines a fixed sequence of steps for answering the query.

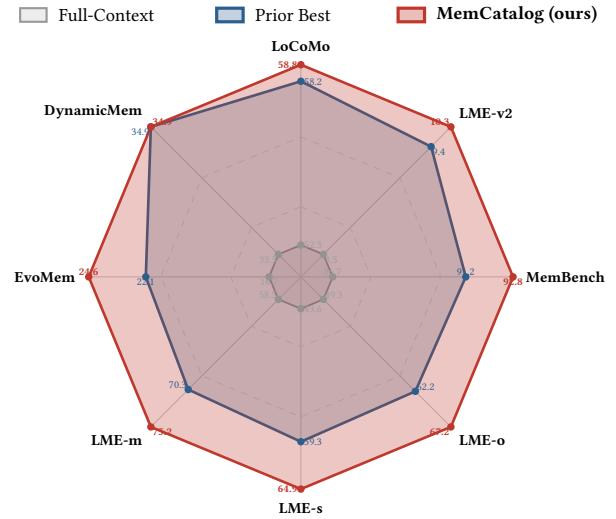


Figure 2: Per-benchmark accuracy on 8 long-term memory benchmarks. Each axis uses its own linear scale: Full-Context maps to the innermost ring, MemCatalog to the outermost regular octagon, and Prior Best falls at its position between them, so that within-benchmark differences among the three systems are directly visible. Raw accuracy (%) is annotated at every vertex. The prior-best system varies per benchmark (MemBench: Mem0; LME-v2: LongMemEval-Fw; the remaining six: Zep), matching Table 2.

- **Second-Order Memory** (Section 5). To address the performance challenge, MemCatalog logs which catalog entries appeared in answers that two independent samples agreed on, and ranks those entries higher for subsequent queries. This is a workload feedback loop, analogous to self-tuning indexes in classical databases [5], applied to agent memory for the first time.
- **Cost-Aware Executor** (Section 6). To address the efficiency challenge, MemCatalog assigns different LLM call budgets to different query patterns: one call for simple lookups, up to three for hard aggregation or comparison queries. It further stops early when two samples already agree, using agreement as a signal that additional calls are unnecessary.

EXAMPLE 1. Alice asks her agent “How many of my events involve outdoor activity?” after six earlier questions. A full-context baseline scans the whole history and answers 5, miscounting evidence scattered across sessions, while the correct answer is 3. A graph memory retrieves six related nodes but never evaluates the condition for each event, and the model guesses 4. MemCatalog routes the query to AGGREGATE (aggregation). The second-order memory promotes event entities that appeared in answers to earlier questions, so the operator reads the relevant events first. It evaluates each candidate with an explicit yes/no judgment and answers 3. The cost-aware executor draws two independent samples; both return 3, so it skips the third sample and commits after two LLM calls. Table 5 gives the full trace.

Contributions. We summarize our contributions as follows.

(1) We formulate agent memory as a data management problem and build MemCatalog, the first memory system that provides a typed catalog, a query planner, and specialized operators for LLM agent memory (Sections 3 and 4).

(2) We introduce *Second-Order Memory*, a workload feedback loop that improves memory quality over time for each user without supervision (Section 5), and a *Cost-Aware Executor* that reduces LLM calls by half while maintaining accuracy (Section 6).

(3) We evaluate MemCatalog on eight long-term memory benchmarks and eight LLM backbones. MemCatalog matches or improves the strongest prior system on every benchmark, by an average of 2.6 points and up to 5.6 points on LME-s, while reducing LLM calls by 50.7% relative to a Uniform- $K=3$ reference (Section 7).

2 Preliminaries

This section defines the concepts and notation used throughout the paper.

2.1 Interaction History and Memory Query

Interaction History. An LLM agent interacts with a user across a sequence of *sessions* $\mathcal{H} = \{s_1, \dots, s_N\}$, where $s_i = (t_i, \text{txt}_i)$ is a timestamp t_i paired with a natural-language transcript txt_i of mixed user and assistant turns. Sessions are read-only after they close, and $\mathcal{H}$ only grows over time. In our benchmarks a single user's history reaches up to 1.5M tokens spread over tens of sessions [21, 39], well beyond any current context window.

Memory Query. A memory query is a triple $q = (Q, C, \tau)$, where Q is a natural-language question, C is an optional set of answer choices (empty for open-ended queries), and τ is the timestamp at which the query is issued. The agent must return an answer $\hat{a}$ derivable from the history available at that moment, $\mathcal{H}_{\leq \tau} = \{s_i : t_i \leq \tau\}$. Answers are scored against a reference answer, with accuracy or F1 computed on $\hat{a}$ (Section 7.1).

2.2 Problem Statement

Given a stream of queries $q_1, q_2, \dots$ over an evolving $\mathcal{H}$, a *memory management system* produces answers $\hat{a}_1, \hat{a}_2, \dots$ under three simultaneous objectives.

- (O1) **Accuracy.** Maximize the fraction of $\hat{a}_t$ that match the gold answer for q_t .
- (O2) **Bounded Cost.** Keep the per-query LLM cost $\text{cost}(q_t)$ bounded as $|\mathcal{H}|$ grows, so the system scales to long histories and heavy usage.
- (O3) **Improvement over Time.** Later queries on the same $\mathcal{H}$ should benefit from the answers to earlier ones.

Prior memory systems focus primarily on (O1) [7, 26, 29, 39]: they report aggregate accuracy across queries, without measuring or controlling cost or per-user adaptation. MemCatalog treats (O2) and (O3) as explicit objectives, enforcing each by a dedicated component: the cost-aware executor for cost (Section 6) and the second-order memory for per-user improvement (Section 5).

2.3 How an Agent Answers a Query

Regardless of the specific memory system, an LLM agent processes each query along two paths. A *write path* fires when a session closes,

converting s_i into the system's internal representation and appending it to a memory store $\mathcal{M}$. Systems differ here in representation: long-context prompting keeps s_i verbatim, RAG embeds chunks, Mem0 extracts facts, and Zep builds graph edges. A *read path* fires per query and proceeds in three common steps: (R1) retrieve a candidate set from $\mathcal{M}$, (R2) pack candidates into a context block that fits the LLM's window, and (R3) reason over that block to produce $\hat{a}_t$.

Two observations about this structure motivate MemCatalog. First, $\text{cost}(q_t)$ is dominated by R3, and drawing multiple independent answers (Self-Consistency [36]) multiplies it on every query regardless of whether the query needed the extra samples. Second, prior systems run the same R1–R3 pipeline for every query and retain nothing from the outcome. Neither what Q_t asks for nor the history of the user's earlier queries influences how the next one is served.

2.4 Three Query Patterns

Natural-language queries against agent memory fall into three patterns that differ in the computation they require. This taxonomy is the entry point for the planner and operator library of Section 4.

- **Point lookup (POINT).** Retrieve a single stored fact, e.g., “Who is my mortgage provider?”. The answer is one value from the catalog and does not require aggregation or comparison.
- **Aggregation (AGGREGATE).** Compute a quantity over multiple records, e.g., “How many of my events involve outdoor activity?”. The answer requires enumerating candidates and applying a predicate to each.
- **Comparison (COMPARE).** Rank or order two or more records on a shared attribute, e.g., “Which lasted longer, the Basel trip or the Zurich conference?”. The answer requires extracting a key from each candidate and reducing to an arg max.

These three patterns parallel common database operations (index lookup, group-by aggregate, order-by comparison), and the LLM's error modes differ across them: point lookups are reliable, aggregation suffers from undercounting when evidence is scattered [14], and comparison suffers from misalignment when values live in distant passages. Prior memory systems collapse all three into one retrieval pipeline, so every query faces every error mode. MemCatalog separates them and gives each its own specialized operator.

3 System Overview

This section traces one query through MemCatalog end to end, so that the roles of the three components are clear before any is specified in detail. Figure 3 shows the architecture: the framework routes each query to a specialized operator, and two optimization layers refine what the operator reads and what it costs.

Query Life Cycle. Given a query q , MemCatalog produces an answer in four steps.

- (1) **Plan.** Route q to one of three patterns (POINT, AGGREGATE, or COMPARE) by matching keywords, with no LLM call (Section 4.2).



Figure 3: Overview of MemCatalog. The framework (catalog, planner, operators) is augmented by two optimization layers: the Second-Order Memory, which closes a workload feedback loop, and the Cost-Aware Executor, which combines pattern-aware budgeting with agreement-based early stopping.

- (2) **Ground.** Assemble relevant catalog entries into a context block. The second-order memory promotes entries that have been useful for this user in the past (Section 5.2).
- (3) **Execute.** Issue LLM calls under the operator’s template. The cost-aware executor decides how many calls the query pattern warrants and stops early when two samples agree (Sections 6.1 and 6.2).
- (4) **Log.** Record which catalog entries appeared in the answer, so that the next query benefits from updated rankings (Section 5.1).

End-to-End Example. We illustrate the full pipeline on Alice, a user whose agent has recorded six months of conversations covering banking, travel, and daily activities.

Step 1: Query Planning. Alice asks “How many of my events involve outdoor activity?” The planner detects the quantifier phrase “how many” and routes the query to AGGREGATE (aggregation). No LLM call is needed.

Step 2: Grounding. The AGGREGATE operator retrieves candidate event entities from the catalog. Because Alice has already asked six questions in this session, the second-order memory has observed that her calendar and activity events frequently appear in accepted answers. These entries are promoted to the top of the candidate list, while unread travel sessions are ranked lower.

Step 3: Execution. The operator enumerates eight candidate events and evaluates each against the predicate “involves outdoor activity” with an explicit yes/no judgment. It produces the count 3. The cost-aware executor draws two independent samples; both return 3, so it skips the third sample and commits the answer after two LLM calls.

Step 4: Logging. The three events that contributed to the answer (e.g., hiking at Uetliberg, cycling by the Rhine, picnic in Bruderholz) are recorded in Alice’s workload log. The next time a query touches these entities, they will be retrieved faster.

Had the same query been issued to a baseline that uses one generic pipeline, the system would scan all sessions uniformly, fail to isolate the eight relevant events, and miscount (the full-context baseline returns 5). MemCatalog avoids this by routing the query to a specialized operator and grounding with workload-informed memory.

How the Three Components Interlock. The three components are not independent. The executor cannot budget calls until the planner has named the query pattern. The operator cannot read a useful candidate set until the second-order memory has ranked the catalog. The second-order memory cannot update until the executor has committed an answer. Removing any one therefore degrades the other two, which Section 7 confirms by ablation.

4 Memory as Managed Data

This section describes how MemCatalog turns unstructured agent memory into managed data. Prior systems process every query through one generic retrieval pipeline, regardless of what the query asks. MemCatalog factors the computation into two parts: a *catalog* that gives the interaction history a structure (Section 4.1), and a *planner* that routes each query to one of three specialized operators (Section 4.2). The overall computation is:

$$\hat{a}_t = \underbrace{\Omega_c}_{\text{Operator}}(q_t, \underbrace{\text{Cat}(\mathcal{H})}_{\text{Catalog}}), \quad c = \underbrace{\rho(q_t)}_{\text{Planner}}, \quad (1)$$

where ρ selects a query pattern $c \in \{\text{POINT}, \text{AGGREGATE}, \text{COMPARE}\}$ and Ω_c is the operator that handles pattern c . We continue to use Alice from Example 1 as the running illustration.

4.1 Typed Catalog

To specialize query processing, the system first needs to know the shape of its data. Raw conversation transcripts do not reveal any shape. The catalog recovers that shape by extracting typed records from sessions.

Typed Records. The catalog $\text{Cat}(\mathcal{H})$ decomposes each session s_i into four record types: *entities* $\mathcal{E}$ (named actors, objects, locations), *events* $\mathcal{V}$ (timestamped occurrences), *relations* $\mathcal{R} \subseteq \mathcal{E} \times P \times \mathcal{E}$ (entity pairs with named predicates), and *attributes* $\mathcal{A} \subseteq \mathcal{E} \times K \times \text{Val}$ ((entity, key, value) triples). For example, a session in which Alice mentions opening a mortgage contributes the entities {Alice, Raiffeisen Basel}, the event (opened mortgage, 2025-03-14), the relation (Alice, has-provider, Raiffeisen Basel), and the attribute (mortgage, rate, 2.4%). What was opaque text now has a visible structure.

On-Demand Extraction. Records are extracted only when needed. When a query first touches session s_i , one templated LLM call returns $(\mathcal{E}_i, \mathcal{V}_i, \mathcal{R}_i, \mathcal{A}_i)$, and the result is cached. Extraction runs once per session and is amortized over all later queries that touch the same session, in the same way a DBMS builds an index once and reuses it across a workload.

Access Paths. Operators need two kinds of lookup: finding which sessions mention a given entity, and finding the value of a given attribute. The catalog exposes one index for each: an entity index $\mathcal{I}_e$ that maps each entity to the sessions it appears in, and an attribute index $\mathcal{I}_a$ that maps each attribute key to its (entity, value, session) records. Operators read memory only through these two indexes, which keeps the storage swappable and the ranking tunable by the second-order memory (Section 5).

4.2 Query Planning and Operators

With structured data in place, the next question is which computation a given query needs. This subsection answers it in two steps: a planning rule that names the query pattern, and an operator library that executes each pattern differently.

Query Planning. The planner ρ uses only the query text and the answer choices:

$$\rho(q) = \begin{cases} \text{AGGREGATE,} & q \text{ matches QuantPhr} \vee C \text{ numeric,} \\ \text{COMPARE,} & \text{else if } q \text{ matches CmpPhr,} \\ \text{POINT,} & \text{otherwise,} \end{cases} \quad (2)$$

where QuantPhr is a fixed set of quantifier phrases (how many, the number of, count) and CmpPhr is a fixed set of comparative markers (more, longer, most). No LLM call or labeled data is needed. For example, “Who is my mortgage provider?” falls through to POINT. “How many of my events involve outdoor activity?” matches how many and routes to AGGREGATE. “Which lasted longer, the Basel trip or the Zurich conference?” matches longer and routes to COMPARE. Because all three operators read the same catalog through the same indexes, each returns a well-formed answer for any query, and POINT serves as the natural fallback for unrecognized patterns.

Operator Library. A point query needs a single value from the catalog, an aggregation query needs to enumerate candidates and count matches, and a comparison query needs to retrieve one value per candidate and compare them. Each operator implements that computation with the catalog indexes:

$$\Omega_{\text{POINT}}(q) = \text{LLM}(\text{template}_{\text{POINT}}(\mathcal{I}_a[k_q])), \quad (3)$$

$$\Omega_{\text{AGGREGATE}}(q) = |\{e \in \mathcal{I}_e[T_q] : \text{LLM}(\text{pred}_q, e) = \text{yes}\}|, \quad (4)$$

$$\Omega_{\text{COMPARE}}(q) = \arg \max_{e \in \{e_1, e_2\}} \mathcal{I}_a[k_q](e). \quad (5)$$

POINT looks up the requested key k_q in $\mathcal{I}_a$ and returns the value directly. **AGGREGATE** enumerates candidate entities of the referenced type T_q from $\mathcal{I}_e$, evaluates each against the query predicate with a bounded yes/no judgment, and emits an integer count. On the outdoor query, it lists Alice’s eight events, tags each, and emits Answer: 3. This makes the count explicit, directly enumerable from structured evidence. **COMPARE** extracts the compared key k_q (here duration) for both candidates from $\mathcal{I}_a$ and presents them side by side, turning an alignment problem over long contexts into a direct comparison of two values.

Remarks. The catalog, the planner, and the operator library form the framework of MemCatalog. Once the query pattern $c = \rho(q)$ is explicit, two further optimizations become possible: the second-order memory can tune the indexes that operators read based on what has worked for each user (Section 5), and the cost-aware executor can set a different budget for each query pattern because it knows which pattern it is serving (Section 6).

5 Second-Order Memory

The framework of Section 4 treats every query independently: the hundredth query for a user is served in the same way as the first. Prior memory systems share this limitation. They observe only the *write path*, which sessions arrived and what was extracted, but never the *read path*, which stored records actually helped answer which queries. As a result, a memory system never learns which parts of a user’s history are valuable.

MemCatalog closes this gap by maintaining a per-user workload log that tracks what memory was useful (Section 5.1), and by using that log to reorganize memory for subsequent queries (Section 5.2). The analogy is a self-tuning database [5, 6]: the system observes the query workload, collects access statistics, and reorganizes its indexes accordingly, all without changing what any query returns.

5.1 Workload Logging

To adapt memory to a user, the system first needs a record of what that user’s queries actually consumed. No existing memory system keeps such a record. This subsection describes what MemCatalog logs and how the log stays compact.

What Is Logged. For each user u , MemCatalog maintains a compact workload log

$$\mathcal{W}_u = \{(q_t, \rho(q_t), E_t^+, \tau_t)\}_{t \leq T}, \quad (6)$$

where τ_t is the timestamp of the query and $E_t^+ \subseteq \mathcal{E}$ is the set of catalog entities that appeared in the accepted answer $\hat{a}_t$ or its supporting evidence. The log captures three signals. The sequence of query patterns $\{\rho(q_t)\}$ describes what the user asks. The entity sets $\{E_t^+\}$ record what memory answers it. Sessions left unread beyond a staleness horizon indicate what memory is unused. After Alice’s first six questions, her log records a POINT-heavy pattern mix over banking and scheduling topics, with frequent hits on Raiffeisen Basel and her calendar events, and no reads on her older travel sessions.

Compact Statistics. The log is statistical and compact. For each catalog entity e , MemCatalog tracks only three running numbers: an issue count $n(e)$ (how often e has been mentioned by any query), a hit count $h(e)$ (how often e appeared in an accepted answer), and the timestamp $\tau^*(e)$ of its most recent hit. Storage is therefore $O(|\mathcal{E}|)$ per user and independent of the number of queries T . Just as a database keeps compact histograms to summarize past SQL workloads, the workload log keeps the distribution of access in a fixed-size summary.

When Is an Answer Accepted? No gold labels exist at deployment time, so acceptance must be defined without supervision. MemCatalog marks an answer as accepted when the executor commits to it under the agreement rule of Eq. (9), i.e., when two independent

samples agree. The log then records only the entities that the operator read in producing that answer. This means the priority score (below) influences which records are retrieved first, never what the operator computes or returns. The feedback loop is a retrieval prior, a way to rank what the operator sees first.

Per-User Isolation. $\mathcal{W}_u$ and every quantity derived from it are strictly per-user. A new user starts from a neutral prior and adapts as questions arrive. There is no cross-user aggregation.

5.2 Workload-Driven Memory Adaptation

A log is useful only if something acts on it. This subsection turns the log into a per-entity priority score and then uses that score to reorganize the catalog, leaving query semantics untouched throughout.

Priority Score. Each catalog entity e carries a priority score that combines three signals:

$$\pi(e) = \underbrace{\alpha \cdot h_r(e)}_{\text{answer hit rate}} + \underbrace{\beta \cdot r(e)}_{\text{recency}} + \underbrace{\gamma \cdot c(e)}_{\text{pattern match}}, \quad (7)$$

where $h_r(e) = h(e)/n(e)$ is the fraction of queries mentioning e whose answer included e , $r(e) = \exp(-\lambda(\tau_{\text{now}} - \tau^*(e)))$ is a recency decay, and $c(e)$ is the fraction of queries mentioning e whose query pattern aligns with e 's record type (for example, an event entity scores higher when past queries were aggregations that enumerate events). The weights $(\alpha, \beta, \gamma) = (0.5, 0.3, 0.2)$ are fixed across all benchmarks and backbones, with direct evidence of usefulness given the dominant share and the remaining two terms acting as tie-breakers. We do not tune them per dataset, and Section 7.6 reports the effect of removing each term.

How the Score Reshapes Memory. $\pi(e)$ is refreshed after every query and drives three physical actions on the catalog, all governed by fixed thresholds:

- *Rank and select.* When an operator retrieves from the indexes $\mathcal{I}_e$ and $\mathcal{I}_a$, the returned entries are ranked by $\pi(e)$. The context block prepended to the operator's prompt draws from this ranking, so a user whose questions focus on banking sees banking entities first, and a user who asks about travel sees travel entities first.
- *Compress cold sessions.* Any session s_i whose entities all fall below a cold threshold π_{cold} is compressed or dropped, freeing space for more relevant content.

By Alice's seventh query, her calendar events sit at the top of the context block while the unread travel sessions have been compressed away. When the AGGREGATE operator enumerates candidates for the outdoor query, the eight relevant events are the first records it reads. The operator itself is unchanged; the workload has reorganized only the catalog it reads.

Remarks. Both actions operate on where memory sits in the ranking, leaving query semantics untouched. The indexes return workload-reweighted results, and every operator benefits without an additional LLM call. A database that observes its SQL workload and reorganizes indexes accordingly becomes faster without changing what any query returns [5, 6]. The same principle applies here: the workload log observes the memory workload, and the priority score reorganizes the catalog accordingly, closing a feedback loop that no prior agent memory system provides.

6 Cost-Aware Execution

A memory system that answers well but spends unboundedly does not scale to long histories and heavy usage. To control this cost while preserving accuracy, MemCatalog introduces a cost-aware executor that exploits the query patterns identified by the planner.

The reason is that different query patterns have different cost profiles. A simple fact lookup (POINT) retrieves one stored value and never benefits from repeated sampling, while an aggregation (AGGREGATE) or comparison (COMPARE) over multiple records genuinely reduces variance with additional samples. Treating all queries the same, as uniform self-consistency does, wastes budget on easy queries and cannot reallocate it to hard ones.

The executor addresses this through two complementary mechanisms: it allocates different budgets to different query patterns (Section 6.1), and it stops spending early when an individual query's answer is already stable (Section 6.2). The two act on different dimensions and their savings compose.

Cost Model. MemCatalog issues LLM calls on both the write path (catalog extraction) and the read path (query answering). Write-path cost is paid once per session and amortizes across queries. The executor targets read-path cost, which dominates. Under fixed per-pattern budgets $(K_{\text{POINT}}, K_{\text{AGGREGATE}}, K_{\text{COMPARE}})$, the total cost over a workload of Q queries with pattern fractions $(p_{\text{POINT}}, p_{\text{AGGREGATE}}, p_{\text{COMPARE}})$ is

$$C = Q \cdot (p_{\text{POINT}} K_{\text{POINT}} + p_{\text{AGGREGATE}} K_{\text{AGGREGATE}} + p_{\text{COMPARE}} K_{\text{COMPARE}}). \quad (8)$$

A Uniform- $K=3$ reference sets $K_{\text{POINT}} = K_{\text{AGGREGATE}} = K_{\text{COMPARE}} = 3$, so $C = 3Q$.

6.1 Pattern-Aware Budgeting

To avoid wasting LLM calls on queries that do not benefit from replication, the executor assigns a budget based on the query pattern, giving each pattern the budget its operator can use productively.

Specifically, the executor sets $K_{\text{POINT}} = 1$ and $K_{\text{AGGREGATE}} = K_{\text{COMPARE}} = 3$. The asymmetry follows from what each operator computes. A POINT query resolves a single stored fact: once (Alice, has-provider, Raiffeisen Basel) is in the context, re-sampling can only re-read the same value, and one call is both necessary and sufficient. AGGREGATE and COMPARE queries compose evidence across multiple records, as when the outdoor count must decide eight independent yes/no judgments, and there sampling genuinely reduces variance, so these patterns retain the full budget.

Substituting these budgets into Eq. (8), the saving against Uniform- $K=3$ is $\Delta C = 2p_{\text{POINT}}Q$. On MemBench with $p_{\text{POINT}} = 0.61$, this alone reduces C from 900 to 532, a 40.9% cut. The saving grows with the share of point queries and costs nothing in accuracy on the queries it economizes.

6.2 Agreement-Based Early Stopping

To further reduce cost on the harder query patterns, the executor detects when additional sampling is unnecessary for a specific query. Some AGGREGATE and COMPARE instances are easy but indistinguishable from hard ones before execution. The executor recovers this distinction at no extra cost by reading the agreement between samples it was going to draw anyway.

Specifically, within the AGGREGATE/COMPARE budget, the executor draws two independent samples p_1, p_2 and measures their token F1 agree($\cdot, \cdot$) $\in [0, 1]$. If the two samples agree above a threshold θ , the query is considered settled and no third sample is drawn:

$$\hat{a} = \begin{cases} p_1, & \text{agree}(p_1, p_2) \geq \theta, \\ \text{Majority}(p_1, p_2, p_3), & \text{otherwise.} \end{cases} \quad (9)$$

On Alice’s outdoor query, both samples enumerate the same eight events and emit Answer: 3, so the agreement is high, the third call is skipped, and the query costs two calls, saving the third. Had one sample returned 3 and the other 4, the low agreement would have triggered the third call, spending it where it can change the outcome. We fix $\theta = 0.6$ for all benchmarks and all backbones.

Combined with pattern-aware budgeting, early stopping drives the total cost on MemBench from 532 to 444, a further 16.5% cut and 50.7% overall against the Uniform- $K=3$ reference.

Remarks. Self-consistency [36] treats disagreement among samples as noise to average away under a fixed budget. The same disagreement, however, also carries information about the query: when two samples agree, the query is easy for the current model and grounding, and a third sample is unlikely to change the outcome. Agreement thus becomes a stopping rule. Adaptive-Consistency [1] and early-stopping self-consistency [17] exploit the same observation within a single query, halting once drawn answers converge, but they must draw at least two samples before any decision is possible. Because MemCatalog knows the query pattern, it can go further: a point lookup cannot benefit from a second sample at all, so the extra sample is skipped entirely for the majority of the workload. The two mechanisms, pattern-aware budgeting across patterns and agreement-based stopping within a pattern, therefore save from disjoint slices and their savings compose.

7 Experiments

We organize the evaluation around eight questions. Exp-1 asks whether MemCatalog outperforms state-of-the-art memory systems across memory abilities. Exp-2 asks whether the advantage survives a change of backbone. Exp-3 asks which query patterns the advantage comes from. Exp-4 asks what the accuracy costs in LLM calls. Exp-5 asks how much each architectural layer contributes. Exp-6 asks whether the second-order memory improves a user over time. Exp-7 asks how accurate the zero-cost planner is and how sensitive the end-to-end result is to its two free parameters. Exp-8 asks how read-path cost behaves as history length grows.

7.1 Experimental Setup

Datasets. We use the eight long-term memory benchmarks in Table 1, chosen to separate three memory abilities that a single benchmark would confound. LoCoMo [21] and LME-v2 [39] test recall over multi-session dialogue. LME-o, LME-s, and LME-m [39] share one query style and scale the surrounding history by nearly two orders of magnitude, which lets history length be varied as a controlled factor. EvoMem [38] and DynamicMem [40] issue many questions per user over an evolving state, the only setting in which per-user adaptation becomes observable. MemBench [34] labels every one of its 300 queries with the query pattern it requires, and it

Table 1: The eight benchmarks, grouped by the memory ability they stress. History length spans two orders of magnitude, which lets us separate gains that come from structure from gains that come from context length.

Benchmark	Memory ability	History
LoCoMo [21]	Multi-session dialogue	~9K
LME-v2 [39]	Entity-centric recall	~115K
MemBench [34]	Typed query patterns	~100K
LME-o [39]	Long-horizon retention	~40K
LME-s [39]	Long-horizon retention	~115K
LME-m [39]	Long-horizon retention	~1.5M
EvoMem [38]	Evolving user state	~50K
DynamicMem [40]	Evolving user state	~127K

is therefore the one benchmark on which a per-class decomposition is possible.

Baselines. We compare against seven systems in two groups. (1)

Prompting Baselines. **Full-Context** [18] places the entire history in the context window, and **RAG** [16] retrieves top- k passages by dense similarity. (2) *Memory Systems.* **MemGPT** [26] pages facts between context tiers. **Mem0** [7] consolidates facts into a dictionary through an ADD/UPDATE/DELETE loop, and **Mem0^g** adds entity relations. **Zep** [29] maintains a temporal knowledge graph. **LongMemEval-Fw** [39] applies a three-stage indexing, retrieval, and reading pipeline. All seven are run from their released implementations under the same backbone, which leaves the memory design as the sole source of any difference.

Evaluation Metrics. Multiple-choice benchmarks are scored by exact match. The remaining benchmarks are scored by F1 against the gold answer, normalized at the phrase level for LME-v2 and at the token level elsewhere. Phrase-level normalization credits a prediction only when it reproduces the gold phrase, so absolute F1 on LME-v2 sits far below the token-level scores, and the two scales are not comparable; within LME-v2 the ranking across systems is unaffected. Higher is better throughout, and every reported number is an accuracy on the benchmark’s own scale.

Backbone LLMs and Settings. Unless stated otherwise all systems run on GPT-5.5. Exp-2 additionally evaluates three further closed-source backbones [3, 11, 24] and four open-source backbones, selected to span a wide capability range at two deployment tiers. MemCatalog is implemented in Python against an OpenAI-compatible client. Every hyperparameter is fixed once and reused across all benchmarks and backbones, with no per-dataset tuning.

7.2 Exp-1. Accuracy Across Three Memory Abilities

We compare MemCatalog against all seven baselines on the eight benchmarks of Table 1, grouped by the memory ability each one stresses. Table 2 gives the comparison. MemCatalog matches or improves the strongest prior system on every benchmark, and the margin varies systematically with the memory ability the benchmark stresses.

Table 2: Accuracy across eight long-term memory benchmarks. MemCatalog matches or improves the strongest prior system on every benchmark, and the margin is widest on the long-horizon and state-inference groups where a single retrieval pass is least sufficient.

Method	Conversational memory			Long-horizon memory			Personalized state	
	LoCoMo	LME-v2	MemBench	LME-o	LME-s	LME-m	EvoMem	DynamicMem
<i>Prompting baselines</i>								
Full-Context [18]	52.3	4.5	86.7	49.3	43.6	58.5	16.7	33.5
RAG [16]	53.1	4.9	87.8	50.8	45.1	59.1	17.3	34.1
<i>Memory systems</i>								
MemGPT [26]	54.5	6.3	89.7	54.1	48.4	61.1	18.0	34.3
Mem0 [7]	56.4	7.2	91.2	58.3	54.2	65.2	20.4	34.5
Mem0 ^g [7]	57.1	7.5	90.8	60.1	56.7	67.1	21.2	34.7
Zep [29]	<u>58.2</u>	8.1	90.4	<u>62.2</u>	<u>59.3</u>	<u>70.3</u>	<u>22.1</u>	<u>34.9</u>
LongMemEval-Fw [39]	57.3	<u>9.4</u>	90.1	61.7	58.8	69.4	21.6	34.6
<i>Ours</i>								
MemCatalog	58.8	10.3	92.8	67.2	64.9	75.2	24.6	34.9

Long-Horizon Memory. The margin is widest on the three Long-MemEval variants, where it reaches 4.9 to 5.6 points, and it does not shrink as history grows by nearly two orders of magnitude. The reason lies in what each system hands the model. A longer history enlarges the candidate set that similarity retrieval returns, so prior systems pass the model more passages and leave it to locate, order, and align the relevant facts inside the context window, the regime in which long-context models are known to undercount and misorder [14, 18]. MemCatalog reads the fields a query needs from the catalog indexes, so the context block stays small and already structured, and the model only performs the reasoning step. The size of that block is set by the query pattern, independent of $|\mathcal{H}|$, which is why the margin persists at the largest scale.

Conversational Memory. The margin is narrowest on this group, at 0.6 points on LoCoMo and 1.6 on MemBench. These benchmarks ask mostly for facts stated in one place, a query pattern that a single retrieval pass already serves well, so there is less headroom for an operator to recover. The result on LME-v2 is 10.3 against 9.4 for the strongest baseline, a 9.6% relative improvement on a strict phrase-level F1 scale on which every system scores in the single digits. What distinguishes MemCatalog on this group is that the accuracy and the cost advantage hold together: on MemBench it reaches the highest accuracy while issuing fewer LLM calls than the uniform reference, as Exp-4 shows.

Personalized State. The two state-inference benchmarks separate the systems least in absolute terms, with all seven baselines falling within 5.4 points on EvoMem and 1.4 points on DynamicMem. These benchmarks issue many questions per user, and a system that treats each query independently answers the tenth exactly as it answered the first, which is what prior designs do: they record what arrives on the write path and never what succeeded on the read path. An aggregate accuracy therefore averages over query position, the dimension on which the second-order memory acts, and hides any per-user improvement. Exp-6 conditions on query position and recovers the effect that the aggregate hides.

Table 3: Accuracy on MemBench across backbones. Best prior is the strongest of the seven baselines of Table 2 re-run on that same backbone. The gain is smallest where the backbone is strongest, so the structure MemCatalog supplies substitutes for capacity the model does not have.

Backbone	Best prior	MemCatalog
<i>Closed-source</i>		
GPT-5.5	91.2	92.8
Claude-3.5-Sonnet	87.4	91.4
Gemini-1.5-Pro	85.6	89.7
GPT-4o	82.8	88.7
<i>Open-source</i>		
DeepSeek-V4	82.7	88.5
Llama-3.1-70B	79.8	86.5
Qwen3-14B	77.4	85.1
Qwen3-8B	73.4	81.3

Finding 1: MemCatalog matches or improves the strongest prior system on all eight benchmarks, and the margin is largest where a single retrieval pass is least sufficient, on long histories and on repeated questions from one user.

7.3 Exp-2. Backbone Generalization

A system that supplies external structure should benefit every backbone, and the benefit should be largest where the backbone’s own capacity is most limited. To test this, we hold MemBench and the full MemCatalog configuration fixed and vary only the backbone, over four closed-source and four open-source models spanning a wide capability range. Table 3 collects the comparison.

MemCatalog improves every backbone, which rules out an explanation resting on one model’s inductive bias. The size of the improvement is what varies. It is smallest on the strongest backbone

Table 4: Accuracy by query pattern on MemBench. Every system handles point lookups well, and the systems separate on aggregation and comparison, which is where a specialized operator replaces unstructured reasoning.

Method	POINT	AGGREGATE	COMPARE
<i>Prompting baselines</i>			
Full-Context [18]	90.9	64.4	84.2
RAG [16]	91.8	67.1	85.8
<i>Memory systems</i>			
MemGPT [26]	92.9	68.5	87.0
Mem0 [7]	93.9	70.2	88.1
Mem0 ^g [7]	94.1	71.9	85.4
Zep [29]	94.3	71.2	85.4
LongMemEval-Fw [39]	93.8	70.8	84.7
<i>Ours</i>			
MemCatalog	94.6	78.0	91.2

and largest on the weakest, and this ordering holds within each group separately. This pattern suggests that external structure compensates for capability the backbone lacks. A frontier model already holds more history in context and counts over it more reliably, so the incremental gain is smaller, but the gain remains positive even on the strongest backbone. The practical consequence is that MemCatalog widens the set of backbones on which long-term memory is usable, which matters most where frontier inference per query is not affordable.

Finding 2: The advantage of MemCatalog is not backbone-specific, and it grows as backbone capability falls, with external memory structure substituting for internal model capacity.

7.4 Exp-3. Per-Pattern Accuracy Decomposition

Exp-1 established that MemCatalog leads overall. To locate the source of that lead, we decompose MemBench accuracy by the pattern each query requires, which MemBench labels directly. Table 4 breaks the accuracy out by query pattern.

The decomposition reveals where the advantage originates. MemCatalog leads on all three query patterns, and the gains concentrate on aggregation and comparison, where a specialized operator replaces unstructured reasoning. The separation appears on these two classes because they require the model to compose evidence across multiple records.

Aggregation. Counting fails for reasons that a better retriever cannot fix. When candidate items are scattered through a long context, models miscount even after retrieving every relevant item [14]. Prior systems retrieve passages and ask for a count, which leaves the count implicit inside unstructured generation. MemCatalog makes it explicit, enumerating candidates, deciding each one against the predicate, then tallying. Each step is bounded, and the total rests on enumerable evidence with an exact count.

Table 5: One aggregation query traced through three systems. The two baselines fail differently, and each failure is removed by a different layer of MemCatalog.

System	Retrieval and reasoning	Calls	Answer
Full-Context	count left implicit	1	5
Zep [29]	predicate unresolved	1	4
<i>MemCatalog, layer by layer</i>			
	Planner routes on the quantifier marker		
	Second-order memory promotes this user’s resolved entities		
	Operator decides each candidate, then tallies		
	Executor withholds the third call on agreement		
MemCatalog	explicit per-item decision	2	3

Comparison. Comparison fails through misalignment. The model must locate the compared attribute in two separate passages and align them before it can order them, and that alignment is what breaks on long histories. MemCatalog performs the alignment outside the model, reading the attribute from the typed index for each candidate and placing both values adjacent in the prompt. The remaining task is an ordering over two values.

Both failure modes live in how the model *processes* retrieved evidence, which is why improving retrieval alone does not close the gap and why the computation has to follow from the query pattern.

To make the two failure modes concrete, Table 5 traces one aggregation query through MemCatalog and two baselines. Both baselines had the relevant material available, and in neither case did retrieval fail. The prompting baseline miscounts evidence spread across sessions, and the graph memory retrieves topically related nodes without resolving the predicate the query asks about. MemCatalog removes the two failures with different layers. The planner selects the AGGREGATE pattern through Eq. (2), the second-order memory promotes the entities this user’s earlier queries resolved, the operator records an explicit decision per candidate before tallying, and the executor withholds its third call once two samples agree.

Finding 3: The accuracy advantage concentrates on aggregation and comparison, where a specialized operator replaces unstructured reasoning over scattered evidence.

7.5 Exp-4. Cost Analysis

Accuracy bought with extra LLM calls is an artifact of the budget, and the architecture should be evaluated separately. To separate the two, we measure the LLM calls each system issues per query on MemBench and place cost and accuracy on the same axes. Cost is counted on the read path for every system, so a system that spends a retrieval or rewriting call before generating is charged for it. Write-path extraction is a one-time per-session cost that every structured system amortizes over the queries touching that session, including the on-demand extraction of MemCatalog, and it is excluded uniformly for all systems. We additionally include two

Table 6: Calls per query on MemBench. The two MemCatalog rows share an accuracy column, which isolates early stopping as a pure cost mechanism and attributes the accuracy to the framework and not to the budget.

Configuration	Calls per query			Total	Acc.
	POINT	AGGREGATE	COMPARE		
Uniform, one call	1.00	1.00	1.00	300	86.7
Uniform, three calls	3.00	3.00	3.00	900	89.1
MemCatalog, budget only	1.00	3.00	3.00	532	92.8
MemCatalog	1.00	2.30	2.18	444	92.8

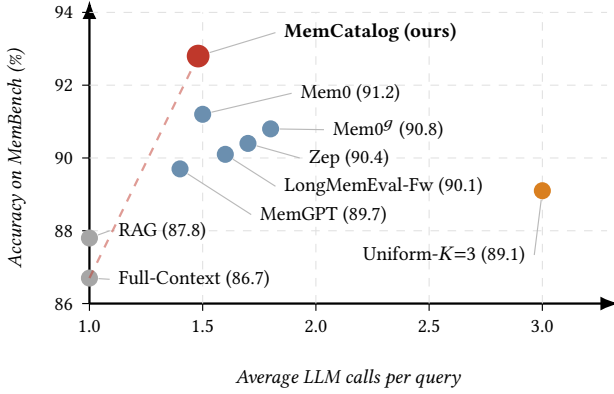


Figure 4: Cost-accuracy Pareto on MemBench (GPT-5.5, $n=300$). Each point is a system's average LLM calls per query (x-axis) versus its accuracy (y-axis). MemCatalog occupies the upper-left corner: strictly higher accuracy than every prior memory system and strictly lower cost than the Uniform- $K=3$ reference. No prior system Pareto-dominates MemCatalog.

reference configurations that fix the sampling budget uniformly at one call and at three calls per query, which isolates the effect of *allocating* a budget from the effect of its *size*.

Figure 4 gives the result. MemCatalog occupies the upper-left region, where no evaluated system attains both lower cost and comparable accuracy. The two groups miss it for different reasons. Prior memory systems are inexpensive and lose accuracy on the queries a single pass cannot answer. The uniform three-call reference recovers part of that accuracy and remains dominated, because it spends its extra samples on the point lookups that never required them.

Table 6 separates the two mechanisms behind this position. Pattern-aware budgeting acts across queries, assigning the single-call budget to the point lookups that form the majority of the workload. Agreement-based early stopping, the rule of Eq. (9), acts within the remaining queries and stops early on those whose two samples already agree. The first mechanism draws from the easy pattern and the second from the easy instances of the hard patterns, and the slices are disjoint, which lets the savings compose. The accuracy column settles what the savings cost: it is unchanged between the two MemCatalog configurations, so early stopping

Table 7: Removing one component at a time. Each layer contributes on a distinct axis, accuracy for the framework, accuracy under repeated queries for the second-order memory, and cost for the executor, confirming the three are complementary.

Configuration	MemBench		LME-v2	
	Acc.	Calls	Acc.	Calls
MemCatalog	92.8	444	10.3	602
<i>Framework</i>				
w/o planner	89.1	900	8.8	903
w/o operators	87.9	900	8.3	903
w/o catalog	86.7	900	7.6	903
<i>Second-order memory</i>				
w/o workload log	92.1	444	8.6	602
w/o priority ranking	92.5	444	9.3	602
w/o session eviction	92.8	444	10.1	651
<i>Cost-aware executor</i>				
w/o pattern-aware budget	92.5	756	10.3	903
w/o early stopping	92.8	532	10.3	723

removes 88 of 532 calls without giving up a single answer, and the accuracy of MemCatalog is attributable to the framework.

Finding 4: MemCatalog is Pareto-optimal against every evaluated system, and early stopping lowers its cost at unchanged accuracy, which attributes the accuracy to the framework, with early stopping purely reducing cost.

7.6 Exp-5. Ablation Study

The three layers of MemCatalog were motivated by three different obstacles. This experiment confirms that each layer contributes independently by removing one component at a time and measuring both accuracy and cost on MemBench and LME-v2, a multiple-choice benchmark and an open-answer one. Table 7 collects the ablations.

Each layer contributes on a distinct axis, confirming they are complementary. Removing any framework component costs accuracy and simultaneously removes the pattern that the budget depends on, so cost rises to the uniform setting. Within the framework the ordering is informative. Removing the catalog is worse than removing the planner, since a planner that routes correctly still has nothing typed to read. The second-order memory contributes most on LME-v2, where a user's repeated questions benefit from read-path feedback, and its three components are not equally active on both benchmarks: session eviction leaves MemBench untouched, whose histories stay within the cache, while on LME-v2 it saves 49 calls. In the executor the two mechanisms differ in kind. Pattern-aware budgeting lowers cost on both benchmarks and also contributes 0.3 points on MemBench, since a point lookup that is sampled three times can be outvoted on the one answer the index already supplies. Agreement-based early stopping changes

Table 8: Accuracy against the position of a query in a user’s own sequence, evaluated on users with ≥ 10 queries so that all four buckets are populated. Only MemCatalog improves as a user keeps asking. On DynamicMem it starts below the static rankers in the first bucket, where its workload log is still empty, and overtakes them once the log carries signal. Absolute levels differ from Table 2, which averages over all users.

Method	Query position for one user			
	1–3	4–6	7–9	10+
<i>EvoMem</i>				
Full-Context	16.7	17.0	16.8	16.5
Mem0 [7]	20.1	20.5	20.4	20.2
Zep [29]	<u>21.8</u>	<u>22.2</u>	<u>22.1</u>	<u>22.0</u>
MemCatalog	22.2	24.4	26.1	27.8
<i>DynamicMem</i>				
Full-Context	28.7	28.8	28.6	28.4
Mem0 [7]	<u>29.4</u>	29.8	29.6	29.3
Zep [29]	29.9	<u>30.0</u>	<u>29.9</u>	<u>29.8</u>
MemCatalog	29.1	30.3	31.7	32.8

no accuracy figure on either benchmark and is a cost mechanism alone.

Finding 5: Each layer contributes on a distinct axis, accuracy for the framework, accuracy under repeated queries for the second-order memory, and cost for the executor, confirming they are complementary.

7.7 Exp-6. Accuracy by Query Position

The second-order memory predicts a specific and falsifiable behavior. Accuracy on one user should rise as that user asks more questions, and no system without a read-path feedback loop should show the same rise. To test this we restrict to users with at least ten queries, so that all four position buckets are populated, condition on the position of a query within its own user’s sequence, and report accuracy per position bucket, holding backbone, prompt, and benchmark fixed so that position is the only variable. Table 8 bins the accuracy by position. Because the evaluation population is a subset of the full test set, the absolute accuracy levels in each column differ from Table 2; the within-table ranking and the monotone trend are the subject of this experiment.

The predicted separation appears on both benchmarks. MemCatalog improves monotonically across position buckets, gaining 5.6 points on EvoMem and 3.7 on DynamicMem from the first bucket to the last, while no baseline moves by more than 0.5 points in either direction and none shows a monotone trend, since none of them records which stored entries produced accepted answers and none has a signal that could vary with position. The starting point is equally informative. In the first bucket the workload log is still empty, so MemCatalog has no second-order signal yet and begins level with the static rankers, below Zep by 0.8 points on

Table 9: Query-planning quality and parameter stability on MemBench. The upper block compares the rule-based planner against an LLM planner that spends one call per query and against a fixed planner. The lower block varies the two parameters MemCatalog fixes system-wide around their defaults $\theta=0.6$ and $\alpha=0.5$, holding all else equal.

Planner	Plan acc.	Planner calls	End-to-end acc.
Rule-based (ours)	91.4	0	92.8
LLM planner	95.1	1	93.2
Always POINT	61.3	0	87.5
<i>Parameter perturbation</i>			
$\theta \in [0.4, 0.8]$	end-to-end acc. 92.3–92.9		
$\alpha \in [0.3, 0.7]$	end-to-end acc. 92.2–92.8		

DynamicMem. It overtakes them from the second bucket onward and leads by 3.0 points by the last. A mechanism that learns from answered queries should hold no advantage before any query has been answered, and this is the crossover that behavior produces.

The improvement cannot be attributed to the model or the prompt, both of which are identical across buckets. It follows the priority score of Eq. (7), which accumulates evidence that an entity participates in answers this user accepts. By the later buckets the context block handed to the operator is composed largely of entities with a demonstrated history of being queried, where a static ranker still offers entities selected by similarity to the current query alone. This is the read-path signal of Section 5, and Exp-5 confirms that removing the workload log removes the effect.

Finding 6: Only MemCatalog improves as a user continues asking, which isolates read-path feedback as the mechanism and shows that prior systems are structurally unable to exhibit this behavior.

7.8 Exp-7. Planner Accuracy and Parameter Stability

The planner governs which operator handles each query, so its quality directly affects the end-to-end result. This experiment validates its design along two dimensions, planning quality and parameter stability. We measure planning accuracy against the class labels MemBench supplies, comparing the rule-based planner against an LLM planner that spends one call per query and against a fixed planner that assigns every query to POINT. We then perturb the early-stopping threshold and the leading weight of the priority score over wide intervals and re-measure end-to-end accuracy. Table 9 covers both.

Three observations follow from the table. First, the rule-based planner reaches 91.4% planning accuracy at zero LLM cost. Second, planning quality has already saturated with respect to what the system does with it: an LLM planner reaches 95.1%, yet buys only +0.4 end-to-end accuracy for one extra call on every query, which is 300 additional calls against a total of 444, and a gap of 0.4 is no larger than the spread the parameter perturbations below produce. Paying two-thirds more calls for a difference of that size is not a

Table 10: Behavior as history grows on the three Long-MemEval variants. Read-path cost is governed by the query pattern and not by history length, so it stays flat while the prompting baseline grows with the context it must carry.

History	Calls / query	Latency (s)	Accuracy
<i>Full-Context</i>			
~40K (LME-o)	1.0	2.3	49.3
~115K (LME-s)	1.0	5.8	43.6
~1.5M (LME-m)	1.0	47.6	58.5
<i>MemCatalog</i>			
~40K (LME-o)	1.9	3.2	67.2
~115K (LME-s)	1.9	3.4	64.9
~1.5M (LME-m)	1.9	3.5	75.2

trade this workload rewards. Third, the fixed planner locates where the value of query planning actually sits: routing every query to POINT costs 5.3 points end-to-end, an order of magnitude more than separates any two planner variants, so what matters is that the query pattern is chosen at all, and the choice granularity has diminishing returns. The lower block perturbs the two parameters MemCatalog fixes system-wide, the early-stopping threshold θ and the leading weight α of the priority score, across intervals spanning half of each parameter’s usable range. End-to-end accuracy stays within 0.6 points over both, which leaves the reported result independent of any parameter search.

Finding 7: The rule-based planner reaches 91.4% planning accuracy at zero cost, an LLM planner cannot convert better planning into materially better answers, and end-to-end accuracy is stable across wide parameter intervals.

7.9 Exp-8. Scalability Analysis

A memory system is only interesting if its cost survives the growth that motivates it. The three LongMemEval variants share one query style and scale the surrounding history by nearly two orders of magnitude, from 40K to 1.5M tokens, which isolates history length as the varied factor. We measure calls per query, end-to-end latency, and accuracy at each scale against the prompting baseline, whose prompt necessarily carries the history it searches. Table 10 tracks all three.

Cost separates the two systems along the length axis. Calls per query are flat for both, at 1.9 for MemCatalog and 1.0 for Full-Context, so latency is where the difference appears. Latency grows 20.7 $\times$ for Full-Context, from 2.3s to 47.6s, and 1.09 $\times$ for MemCatalog, from 3.2s to 3.5s. At the shortest history MemCatalog is the slower of the two, by 0.9s, because catalog access is a fixed overhead that a single prompting call does not pay. That overhead does not grow, so it is already repaid at 115K, where MemCatalog answers in 3.4s against 5.8s, and at 1.5M it is 13.6 $\times$ faster. What the table shows is a different asymptote, with MemCatalog converging to a flat cost while the baseline grows linearly. Accuracy moves with the difficulty of each variant for both systems, and the ranking does

not: MemCatalog leads at all three scales, by 17.9, 21.3, and 16.7 points, with no erosion as history grows.

The asymmetry is structural and follows from where each system spends. A prompting baseline must carry the history it searches, so its prompt grows with $|\mathcal{H}|$ and its latency grows with it. The read-path cost of MemCatalog is set by the query pattern through Eq. (8), which depends on the shape of the query alone, independent of the size of the store, and adding sessions changes what the catalog contains without changing what a query costs to answer.

Finding 8: Read-path cost is governed by the query pattern, independent of history length, which turns the growth of the store from a linear cost into a constant one and leaves the accuracy advantage of MemCatalog intact at every scale.

8 Related Work

8.1 Agent Memory Systems and Benchmarks

The dominant thread in agent memory research treats memory as a *storage* abstraction. MemGPT [26] and its framework successor Letta [25] draw inspiration from operating-system memory hierarchies. They treat the LLM context window as a fast tier and external stores as slow tiers, with paging between them managed by the agent via function calls. Their contribution is the movement policy across tiers, and the query-time algebra over the stored data remains a single retrieval pass. Mem0 [7] designs a write-time pipeline that extracts, evaluates, and consolidates salient facts through an ADD/UPDATE/DELETE/NOOP loop, and its graph variant Mem0^g adds entity and relation edges. On the read side Mem0 falls back to dense retrieval top- k , without a query planner or any optimization specific to the query pattern. Zep [29], powered by the Graphiti engine, models memory as a temporal knowledge graph over episodes, entities, and communities. It is tuned for retrieval accuracy and latency, but every query still goes through the same retrieval strategy. Further work such as A-MEM [41], AriGraph [2], HippoRAG [13], MemoRAG [28], and hierarchical retrieval methods such as RAPTOR [30] and GraphRAG [10] add autonomous memory evolution, biologically inspired retrieval, and global memory augmentation. Skill-library agents such as Voyager [35] treat learned skills as reusable memory. All of these systems enrich the storage layer, as chronicled by a recent survey [20]. None provides a query planner over typed operators, a cost model that budgets LLM calls by query pattern, or a workload feedback loop. MemCatalog fills these three gaps in Sections 4, 6, and 5.

Long-term memory evaluation has matured in parallel. LoCoMo [21] introduces multi-session dialogue at scale, and LongMemEval [39] covers five memory abilities up to 1.5M tokens while contributing the three-stage pipeline our LongMemEval-Fw baseline follows. Further benchmarks include MemoryBank [45], PerLTQA [9], DialSim [15], and LongBench [4].

8.2 DBMS Foundations for LLM Systems

Two complementary DBMS lines meet in this paper. First, a body of work applies DBMS abstractions to LLM analytics over *typed tabular data*. LOTUS [27] defines the first declarative model for AI-based tabular transformations such as semantic filter, join, top- k ,

and group-by, with gold algorithms and cost-optimized execution plans that achieve up to 1,000× speedup. DocETL [32], UQE [8], and SUQL [19] address related settings for unstructured documents and hybrid structured/unstructured search. These works share the core intuition of MemCatalog, namely to lift LLM operations to a queryable algebra, but their target data is structured (DataFrames, tables). Agent memory is unstructured, its schema is latent in natural language, and its queries are user questions expressed as free text. To our knowledge MemCatalog is the first system to transfer the full query-processing stack (planner, operators, cost model, and statistics) to agent memory.

Second, three founding lines of RDBMS research supply the pillars MemCatalog borrows, namely cost-driven access-path selection [31], extensible operator libraries [12], and workload-driven self-tuning [5, 6] that learned optimizers [22, 23, 42] later extended. The planner, the operator library, and the second-order memory of MemCatalog play these three roles in turn. The adaptation is non-trivial, since the operators reason over natural language, the records are unstructured, and the cost unit becomes an LLM call, but the separation of concerns transfers intact.

8.3 LLM Reasoning Primitives

Chain-of-Thought [37], Self-Consistency [36], ReAct [44], Reflexion [33], and Tree-of-Thoughts [43] form the reasoning substrate on which the operators of MemCatalog build. A line of work reduces the cost of self-consistency by adapting how many samples a query receives, halting once drawn answers agree [1, 17]. These methods decide within a single query and must draw at least two samples before any decision is possible. MemCatalog composes with them from the opposite direction, using the query pattern to decide how much budget a query is eligible for before sampling begins, which removes the extra sample entirely for the class that cannot benefit from it (Section 6).

9 Conclusion

We introduced MemCatalog, a DBMS-style memory management system for LLM agents. MemCatalog organizes agent memory as managed data with a catalog, a zero-cost planner, and three specialized operators, and augments this framework with two optimization layers: a *Second-Order Memory* that lets storage adapt to per-user workload, and a *Cost-Aware Executor* that allocates its call budget by query pattern and by sample agreement. On eight long-term memory benchmarks and eight LLM backbones, MemCatalog matches or improves the strongest prior memory system on every benchmark, by up to 5.6 points on LME-s. On MemBench it further reduces LLM calls by 50.7% relative to a Uniform- $K=3$ reference while attaining higher accuracy, and its second-order memory is the only mechanism among the evaluated systems whose accuracy rises monotonically across successive queries from the same user. The classical DBMS separation of storage, planning, and execution therefore transfers cleanly to agent memory, and each of the three layers contributes measurably. We view MemCatalog as a first step toward treating LLM agent memory as first-class managed data, a setting in which decades of data management results become applicable.

Future work will explore multi-user catalog sharing under privacy constraints, tighter integration of MemCatalog with retrieval-native architectures that pre-index by query pattern, and extensions of the operator library to multi-hop reasoning that currently falls back to AGGREGATE enumeration.

References

- [1] Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. 2023. Let’s Sample Step by Step: Adaptive-Consistency for Efficient Reasoning and Coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. arXiv:2305.11860.
- [2] Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Mikhail Burtsev, and Evgeny Burnaev. 2024. AriGraph: Learning Knowledge Graph World Models with Episodic Memory for LLM Agents. *arXiv preprint arXiv:2407.04363* (2024).
- [3] Anthropic. 2024. The Claude 3.5 Sonnet Model Card. <https://www.anthropic.com/news/claude-3-5-sonnet>.
- [4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2023. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *arXiv preprint arXiv:2308.14508* (2023).
- [5] Surajit Chaudhuri and Vivek Narasayya. 1997. An Efficient Cost-Driven Index Selection Tool for Microsoft SQL Server. In *VLDB*.
- [6] Surajit Chaudhuri and Gerhard Weikum. 2007. Self-Tuning Database Systems: A Decade of Progress. In *VLDB*.
- [7] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Memo0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413* (2025).
- [8] Hanjun Dai, Bethany Yixin Wang, Xingchen Wan, Bo Dai, and Dale Schuurmans. 2024. UQE: A Query Engine for Unstructured Databases. *arXiv preprint arXiv:2407.09522* (2024).
- [9] Yiming Du, Hongru Wang, Zhengyi Zhao, Bin Liang, Baojun Wang, Wanjuan Zhong, Zezhong Wang, and Kam-Fai Wong. 2024. PerLTQA: A Personal Long-Term Memory Dataset for Memory Classification, Retrieval, and Synthesis in Question Answering. *arXiv preprint arXiv:2402.16288* (2024).
- [10] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [11] Google DeepMind. 2024. Gemini 1.5: Unlocking Multimodal Understanding across Millions of Tokens of Context. <https://arxiv.org/abs/2403.05530>.
- [12] Goetz Graefe. 1993. Query Evaluation Techniques for Large Databases. *Comput. Surveys* 25, 2 (1993), 73–169.
- [13] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *arXiv preprint arXiv:2405.14831* (2024).
- [14] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekeshe, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What’s the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654* (2024).
- [15] Jiho Kim, Woosog Chae, Yeonsu Kim, Seokchan Cho, Yoonseo Kim, Kyungjae Kim, Yohan Choi, and Jaegul Choo. 2024. DialSim: A Real-Time Simulator for Evaluating Long-Term Multi-Party Dialogue Understanding of Conversational Agents. *arXiv preprint arXiv:2406.13144* (2024).
- [16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
- [17] Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. 2024. Escape Sky-high Cost: Early-stopping Self-Consistency for Multi-step Reasoning. In *International Conference on Learning Representations (ICLR)*.
- [18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *arXiv preprint arXiv:2307.03172* (2023).
- [19] Shicheng Liu, Jialiang Xu, Wesley Tjanganaka, Sina J. Semnani, Chen Jie Yu, and Monica S. Lam. 2023. SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models. *arXiv preprint arXiv:2311.09818* (2023).
- [20] Junyu Luo et al. 2026. From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms. *arXiv preprint arXiv:2605.06716* (2026).
- [21] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2402.17753.

- [22] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2020. Bao: Learning to Steer Query Optimizers. *arXiv preprint arXiv:2004.03814* (2020).
- [23] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: A Learned Query Optimizer. *arXiv preprint arXiv:1904.03711* (2019).
- [24] OpenAI. 2023. GPT-4 Technical Report. <https://arxiv.org/abs/2303.08774>.
- [25] Charles Packer, Sarah Wooders, et al. 2024. Letta: A Framework for Stateful LLM Agents with Memory Management. Successor of MemGPT; <https://github.com/letta-ai/letta>.
- [26] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023).
- [27] Liana Patel, Siddharth Jha, Melissa Pan, Carlos Guestrin, and Matei Zaharia. 2024. Semantic Operators: A Declarative Model for Rich, AI-based Data Processing. *arXiv preprint arXiv:2407.11418* (2024).
- [28] Hongjin Qian, Zheng Liu, Kelong Mao, Yujia Zhou, and Zhicheng Dou. 2024. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. *arXiv preprint arXiv:2409.05591* (2024).
- [29] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956* (2025).
- [30] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In *ICLR*. arXiv:2401.18059.
- [31] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *ACM SIGMOD*. 23–34.
- [32] Shreya Shankar, Tristan Chambers, and Aditya G. Parameswaran. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. In *ICLR*. arXiv:2410.12189.
- [33] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*. arXiv:2303.11366.
- [34] Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents. In *Findings of the Association for Computational Linguistics: ACL 2025*. arXiv:2506.21605. doi:10.18653/v1/2025.findings-acl.989
- [35] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291* (2023).
- [36] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *ICLR*. arXiv:2203.11171.
- [37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*. arXiv:2201.11903.
- [38] Tianxin Wei, Naveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. 2025. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory. *arXiv preprint arXiv:2511.20857* (2025).
- [39] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv preprint arXiv:2410.10813* (2024).
- [40] Wenya Xie, Shengming Zhou, Zelin Li, Pouya Parsa, Shuang Zhou, Xinheng Ding, Chinmay Arvind, and Guanchu Wang. 2026. DynamicMem: A Long-Horizon Memory Benchmark in Real-World Settings. *arXiv preprint arXiv:2606.22877* (2026).
- [41] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110* (2025).
- [42] Zongheng Yang, Wei-Lin Chiang, Sifei Luan, Gautam Mittal, Michael Luo, and Ion Stoica. 2022. Balsa: Learning a Query Optimizer Without Expert Demonstrations. *arXiv preprint arXiv:2201.01441* (2022).
- [43] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *NeurIPS*. arXiv:2305.10601.
- [44] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*. arXiv:2210.03629.
- [45] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. In *AAAI*. arXiv:2305.10250.